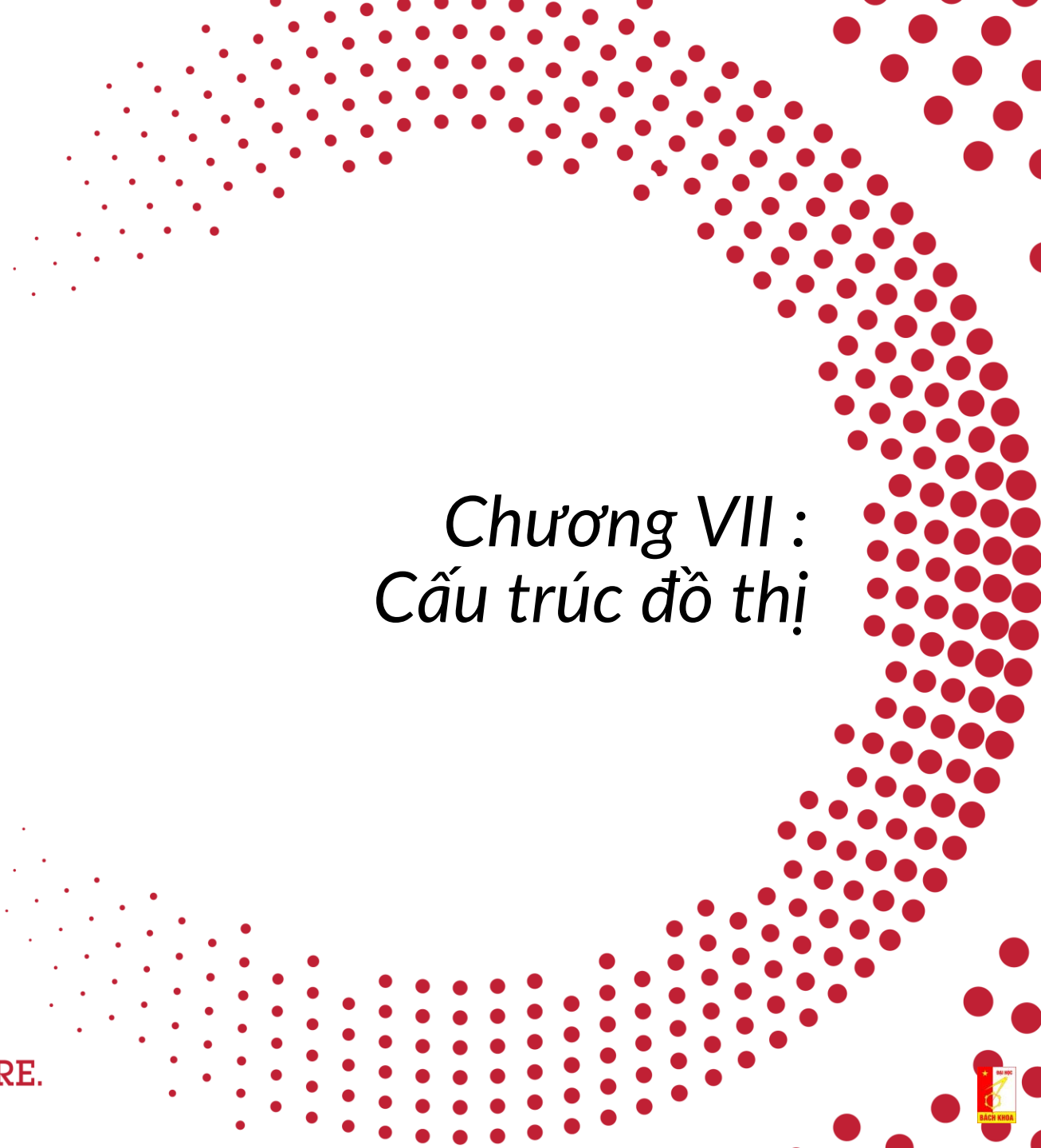


HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT ET2100

ONE LOVE. ONE FUTURE.



Chương VII : Cấu trúc đô thị

ONE LOVE. ONE FUTURE.

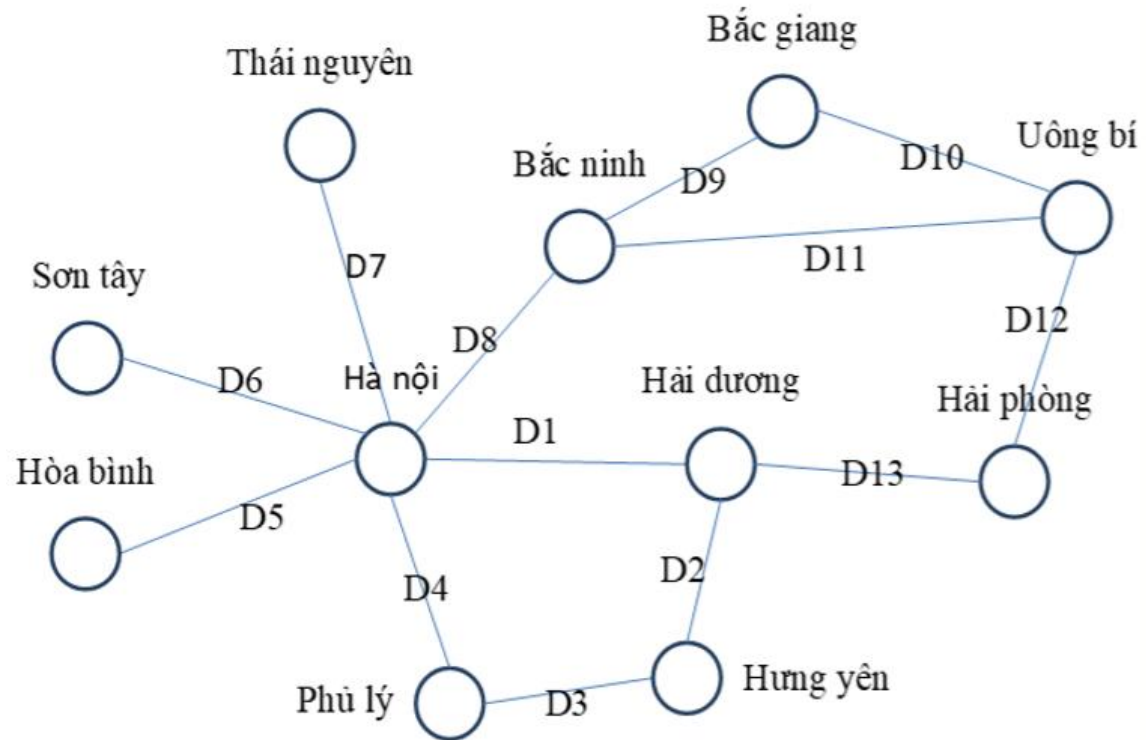
1. Các khái niệm cơ bản (*Basic concepts*)
2. Cài đặt cấu trúc đồ thị (*Graph implementation*)
 - 2.1. Sử dụng cấu trúc tuần tự (ma trận kề)
 - 2.2. Sử dụng cấu trúc móc nối (danh sách kề)
3. Phép duyệt đồ thị (*Graph traversal*)
4. Một số bài toán về đồ thị (*Graph problems*)
 - 4.1. Bài toán tìm đường đi ngắn nhất (*Shortest path search*)
 - 4.2. Bài toán tìm chu trình của đồ thị (*Cycle detection*)
 - 4.3. Bài toán tìm cây khung có giá cực tiểu (*Minimum spanning trees*)

1. Các khái niệm cơ bản

- Đồ thị (graph):
 - là một cấu trúc dữ liệu trừu tượng (abstract) được sử dụng để cài đặt đồ thị trong toán học
 - có thể coi như khái quát của cấu trúc cây, trong đó chỉ có quan hệ giữa cha và con
 - được sử dụng để mô hình hóa bất kỳ tình huống nào trong đó các thực thể hoặc đối tượng có quan hệ từng cặp với nhau

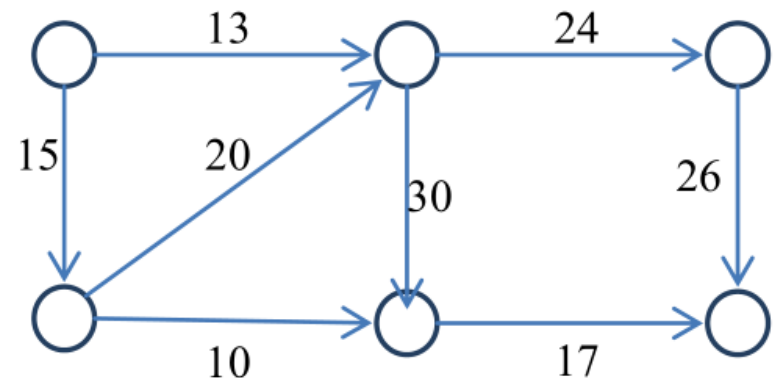
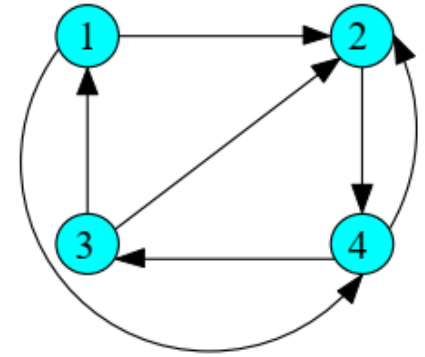
Các khái niệm cơ bản

- Một đồ thị $G = (V, E)$: gồm hai thành phần
 - V : tập các đỉnh (vertices) : $v_0, v_1, \dots, v_{m-1}$
 - E : tập các cung/cạnh (edges) nối các đỉnh: $e_0, e_1, \dots, e_{n-1}$
 - Một cung $e_i = (v_j, v_k)$ là một cặp đỉnh
- Ví dụ:
 - Bản đồ giao thông là một đồ thị



Các loại đồ thị

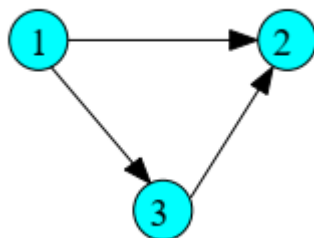
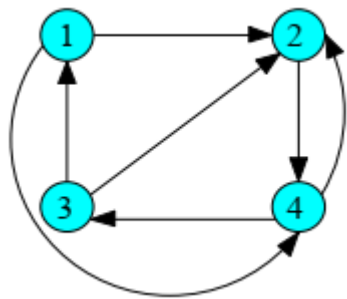
- Đồ thị có hướng (directed graph):
 - Mỗi cạnh đều có một chiều nhất định
 - Trong một cạnh, thứ tự của các đỉnh là quan trọng: $(u,v) \neq (v,u)$
- Đồ thị vô hướng (undirected graph):
 - Trong một cạnh, thứ tự của các đỉnh là không quan trọng $(u,v) \equiv (v,u)$
- Đồ thị có trọng số (weighted graph):
 - Mỗi cạnh mang một giá trị được gọi là trọng số của cạnh đó
- Ngược lại là đồ thị không có trọng số



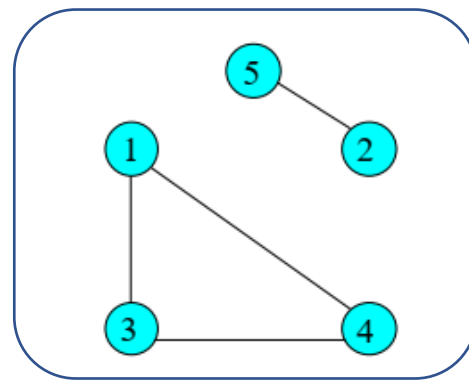
Các loại đồ thị

- Đồ thị liên thông (connected graph)

- giữa hai đỉnh bất kỳ luôn có ít nhất một đường đi

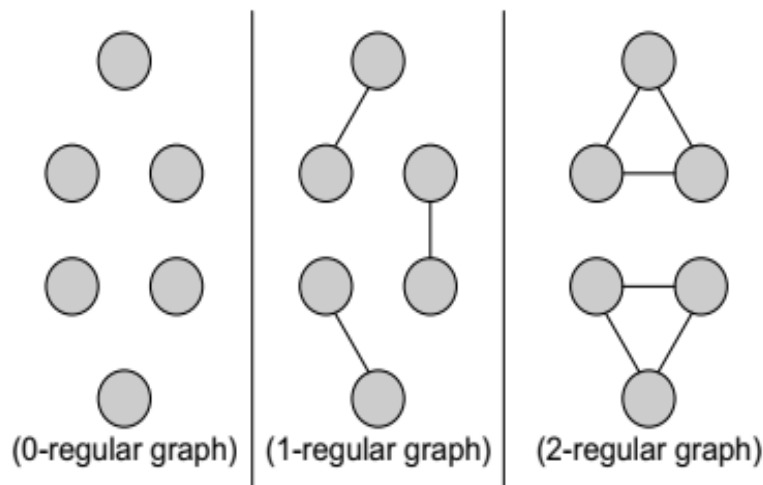


- Đồ thị không liên thông



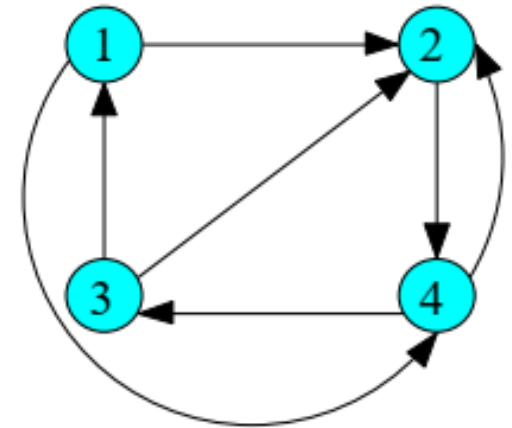
- Đồ thị đều (Regular graph)

- mỗi đỉnh có số đỉnh kề bằng nhau



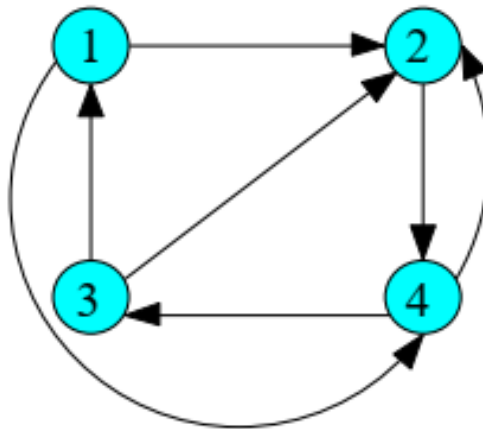
Các khái niệm liên quan

- **Bậc của một đỉnh (degree):**
 - là số cung kề với đỉnh
- Nếu là đồ thị có hướng, một đỉnh có thể có:
 - Bậc trong (in-degree)
 - Bậc ngoài (out-degree)
- Một đỉnh có bậc = 0: đứng biệt lập
- Ví dụ:
 - Đỉnh 1 có bậc 3
 - Đỉnh 1 có bậc trong là 1 và bậc ngoài là 2



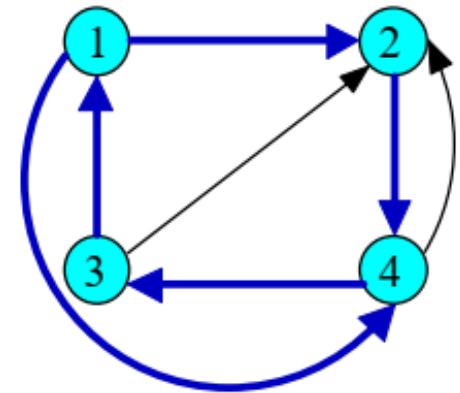
Một số tính chất của đồ thị

- Nếu đồ thị G có m cung $\rightarrow$ Tổng bậc của các đỉnh trong G là $2m$
- Nếu đồ thị có hướng có m cung $\rightarrow$ Tổng bậc trong của các đỉnh, tổng bậc ngoài của các đỉnh đều là m
- Nếu đồ thị G là đồ thị có n đỉnh và m cung thì
 - Nếu G là đồ thị vô hướng $m \leq n(n-1)/2$
 - Nếu G là đồ thị có hướng thì $m \leq n(n-1)$



Các khái niệm liên quan

- Đỉnh lân cận/kề (adjacent vertices) và cạnh kề (adjacent edges)
 - v_j là đỉnh kề của v_i nếu $(v_i, v_j) \in E$.
 - (v_i, v_j) là cạnh kề của v_i và v_j
- Đường đi: $P(v_1, v_k) =$ Dãy các đỉnh $v_1, v_2, \dots, v_k$ mà tồn tại cung (v_i, v_{i+1}) trong đồ thị ($i = 1 \dots k-1$)
- Đường đi đơn: Đường đi với các đỉnh không lặp lại
- Chu trình: Đường đi đơn với đỉnh đầu và cuối trùng nhau
- Độ dài đường đi: số cung trên đường đi



Path : 1, 2, 4, 3, 1, 4

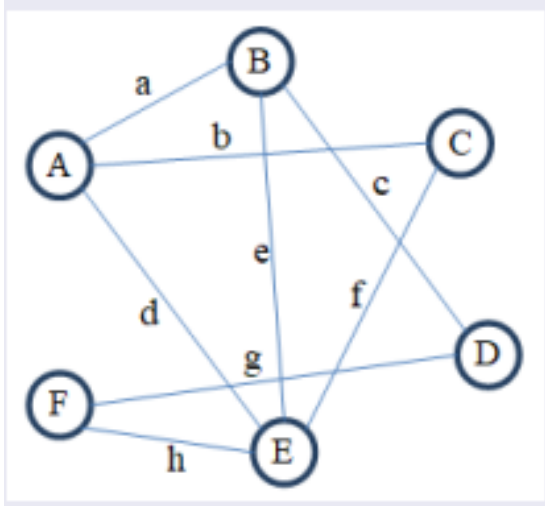
- Duyệt đồ thị: truy cập vào tất cả các đỉnh của đồ thị, mỗi đỉnh đúng một lần.
 - Hai phương pháp duyệt đồ thị: Duyệt theo chiều sâu, Duyệt theo chiều rộng
- Tìm kiếm một đỉnh hoặc một cạnh
 - Tìm kiếm này thường dựa vào các phương pháp duyệt
- Tìm đường đi ngắn nhất
 - tìm đường đi ngắn nhất giữa hai điểm
 - tìm đường đi ngắn nhất giữa một điểm và tất cả các điểm còn lại
 - v.v.

2. Cài đặt cấu trúc đồ thị

- Sử dụng cấu trúc lưu trữ tuần tự
 - Sử dụng cấu trúc mảng để lưu trữ các đỉnh và các cạnh của đồ thị
 - Ưu điểm: tổ chức lưu trữ đơn giản, thao tác cài đặt có tốc độ nhanh do khả năng truy nhập trực tiếp của cấu trúc mảng.
 - Chỉ thích hợp khi kích thước đồ thị nhỏ (khoảng vài chục đỉnh và cạnh).
- Sử dụng cấu trúc lưu trữ móc nối
 - Hạn chế: tốc độ các thao tác chậm hơn so với cách sử dụng cấu trúc tuần tự, do đặc điểm phải truy nhập tuần tự của cấu trúc móc nối.
 - Có thể tăng cường các điểm truy nhập, nhưng cần cân bằng với mức độ phức tạp và lãng phí bộ nhớ.

- Cài đặt đồ thị $G(V,E) : V = (v_0, v_1, \dots, v_{M-1})$, $E = (e_0, e_1, \dots, e_{N-1})$
- Sử dụng:
 - Mảng $AV[M]$: lưu đỉnh của đồ thị (M là số đỉnh của đồ thị)
 - Lưu cạnh:
 - Ma trận **đỉnh kề**: mảng hai chiều $AE[M \times M]$ để lưu trữ các cạnh
 - nếu $e_i = (v_j, v_k)$ thì $AE[j, k] = \text{true}$ (đồ thị không trọng số)
hoặc $= w_{e_i}$ (đồ thị có trọng số)
 - Ma trận **cạnh kề**: cần 2 mảng
 - $E[N]$: lưu các cạnh (nếu đồ thị không có trọng số, giá trị các cạnh là không quan trọng, mảng E có thể bỏ).
 - $AE[M \times N]$: lưu trạng thái các cạnh kề
 - nếu cạnh e_i kề với đỉnh v_j thì $AE[j, i] = \text{true}$.

Biểu diễn đồ thị - ma trận kề



Ma trận đỉnh kề

	A	B	C	D	E	F
A		T	T		T	
B	T			T	T	
C	T				T	
D		T				T
E	T	T	T			T
F				T	T	

Ma trận cạnh kề

	a	b	c	d	e	f	g	h
A	T	T		T				
B	T		T		T			
C		T				T		
D			T				T	
E				T	T	T		T
F							T	T

→ Thường được sử dụng hơn

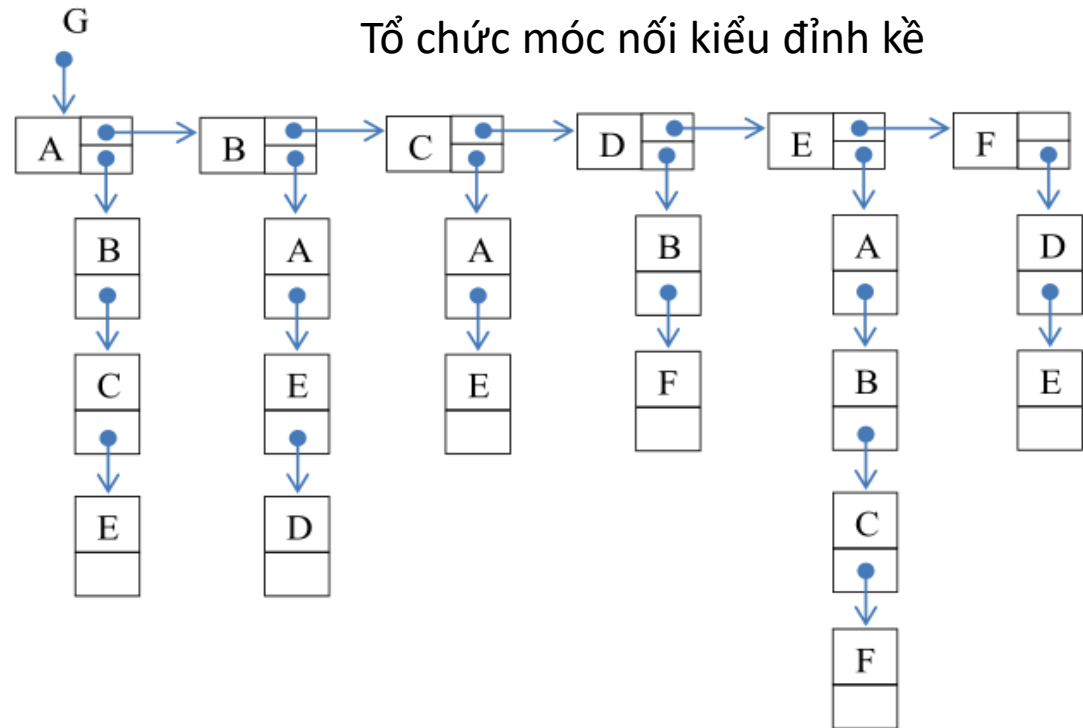
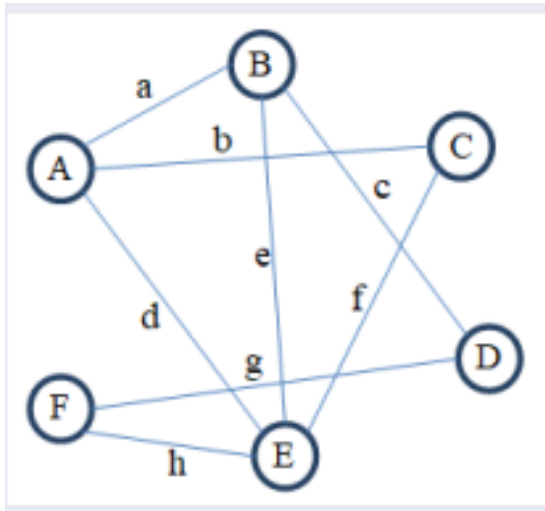
Hạn chế:

+ hạn chế số lượng đỉnh tối đa của đồ thị

+ đồ thị vô hướng --> ma trận đối xứng: lãng phí

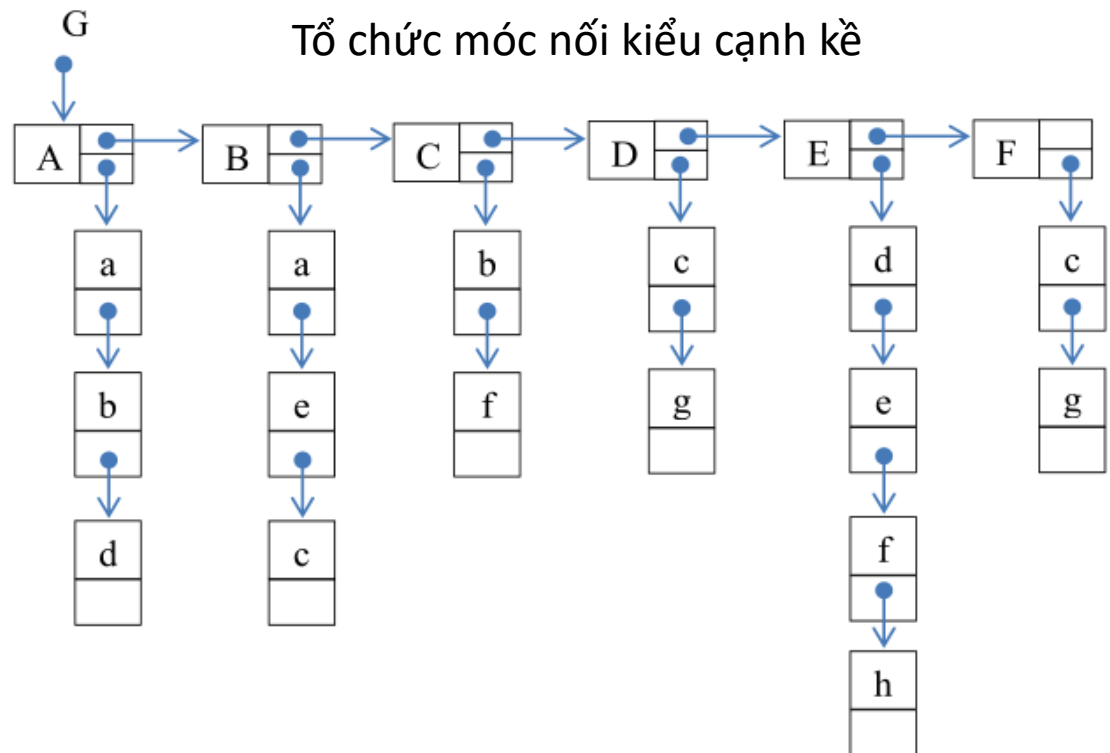
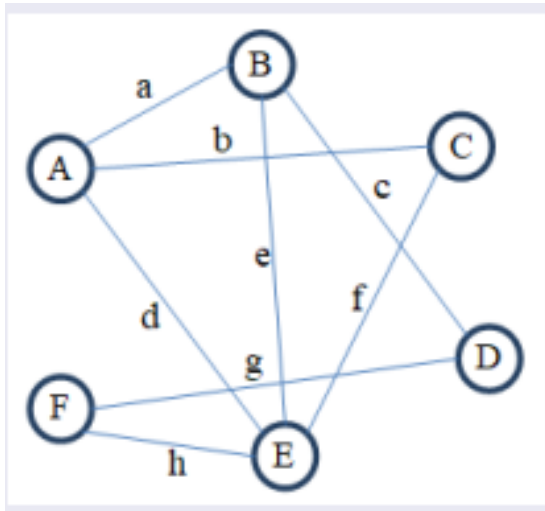
Sử dụng cấu trúc móc nối

- Cài đặt đồ thị $G(V,E) : V = (v_0, v_1, \dots, v_{M-1})$, $E = (e_0, e_1, \dots, e_{N-1})$
 - Tập các đỉnh: biểu diễn bằng một danh sách móc nối (đơn)
 - Tập các cạnh: cũng biểu diễn bằng danh sách móc nối
 - Tổ chức danh sách đỉnh kề: các đỉnh kề với một đỉnh sẽ nằm trong danh sách



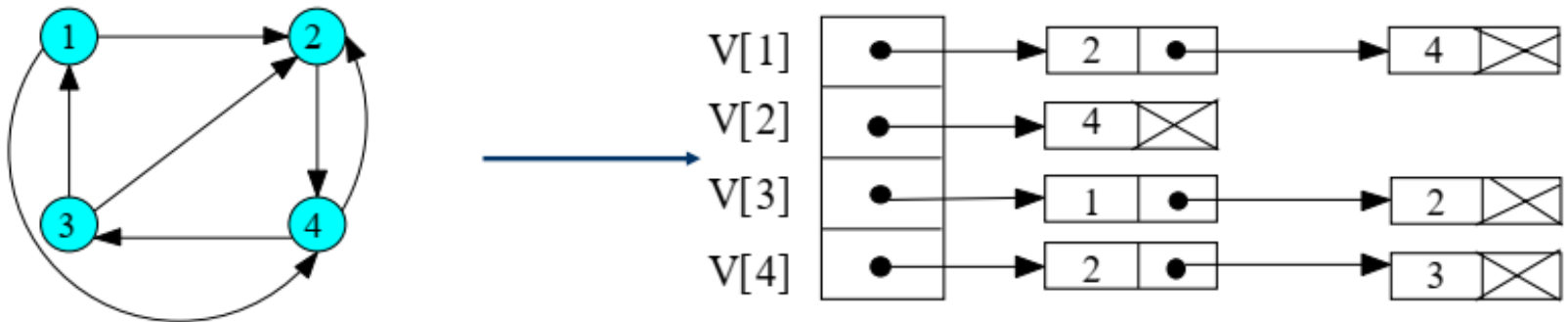
Sử dụng cấu trúc móc nối

- Cài đặt đồ thị $G(V,E)$: $V = (v_0, v_1, \dots, v_{M-1})$, $E = (e_0, e_1, \dots, e_{N-1})$
 - Tập các đỉnh: biểu diễn bằng một danh sách móc nối (đơn)
 - Tập các cạnh: cũng biểu diễn bằng danh sách móc nối
 - Tổ chức danh sách đỉnh kề: các đỉnh kề với một đỉnh sẽ nằm trong danh sách
 - Tổ chức danh sách cạnh kề: các cạnh kề với một đỉnh sẽ nằm trong danh sách



Sử dụng cấu trúc móc nối

- Cài đặt đồ thị $G(V,E) : V = (v_0, v_1, \dots, v_{M-1})$, $E = (e_0, e_1, \dots, e_{N-1})$
 - Tập các đỉnh: biểu diễn bằng một mảng $V[M]$
 - Tập các cạnh: biểu diễn bằng danh sách móc nối
 - Tổ chức danh sách đỉnh kề
 - Tổ chức danh sách cạnh kề



3. Phép duyệt đồ thị

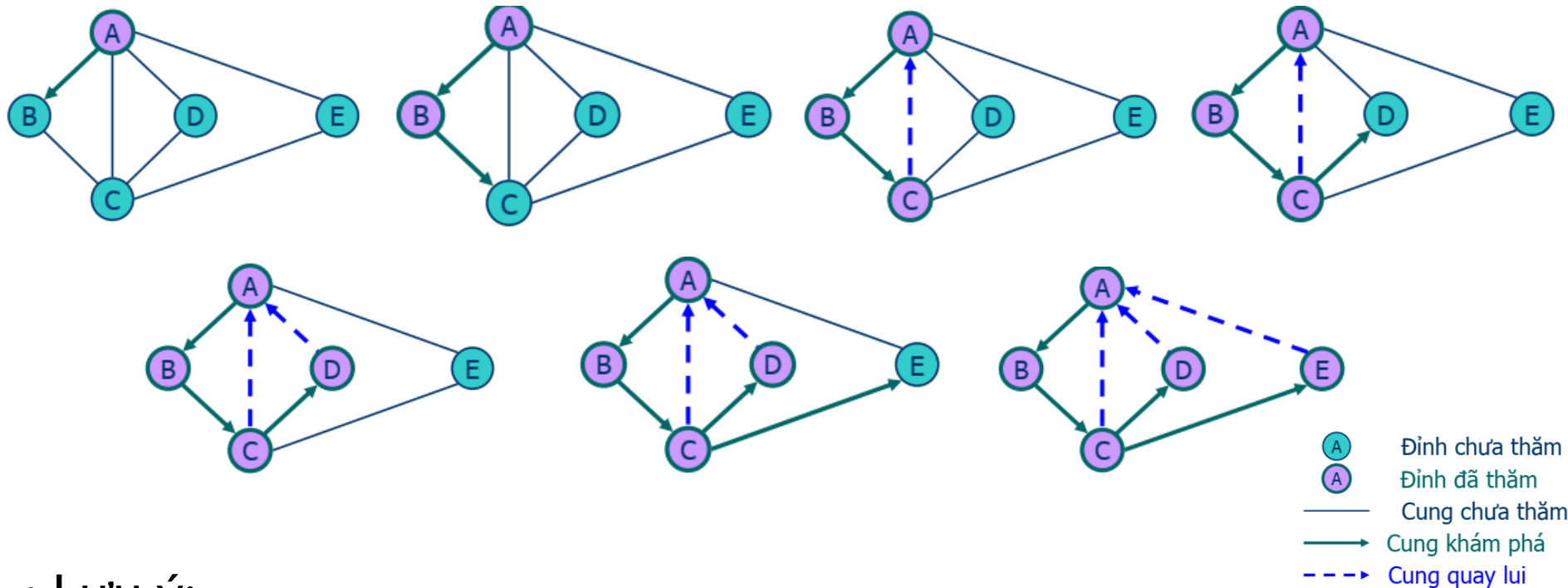
- Cho trước đồ thị $G(V, E)$ và nút xuất phát $v_s \in V$
 - Duyệt tất cả các đỉnh của G , mỗi đỉnh đúng một lần, bắt đầu từ nút v_s .
- Chọn v_s : **cho trước**, chọn ngẫu nhiên, thỏa mãn điều kiện xuất phát nào đó, nút được truy nhập đầu tiên, v.v.
- Có 2 phương pháp:
 - Duyệt theo chiều sâu (Depth First Search)
 - Duyệt theo chiều rộng (Breadth First Search)

- Giải thuật đệ quy :

- Bước 1: $v = v_s$.
- Bước 2: Thăm nút v .
- Bước 3: Với mỗi nút v_a là kề của v mà chưa được thăm, lặp lại bước 2 cho v_a .
- Bước 4 (Điểm dừng): giải thuật kết thúc khi không còn nút nào cần phải thăm.

```
void DepthFirstSearch(G, v) {  
    if (v == NULL) return;  
    visit(v);  
    for each unvisited adjacent vertices  $v_a$  of  $v$   
        // normally sorted by key value  
        DepthFirstSearch(G,  $v_a$ );  
}  
  
void FullDepthSearch(G,  $v_s$ ) {  
     $v = v_s$ ;  
    DepthFirstSearch(G,  $v$ );  
}
```

Duyệt theo chiều sâu



• Lưu ý:

• Đồ thị có tồn tại chu trình:

- cần cập nhật đúng các trạng thái đã thăm/chưa thăm của từng nút trong quá trình duyệt để tránh rơi vào vòng lặp vô hạn.

• Đồ thị có các nút cô lập:

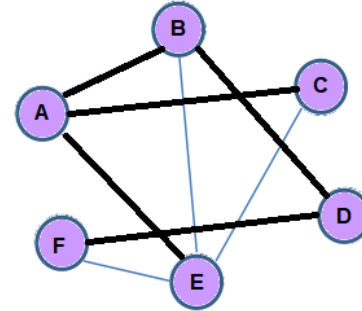
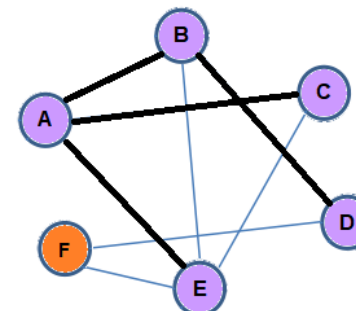
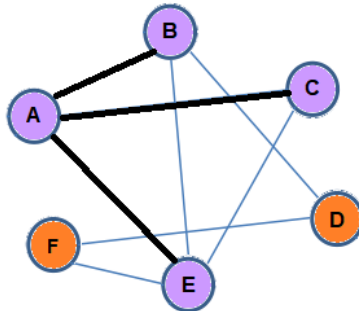
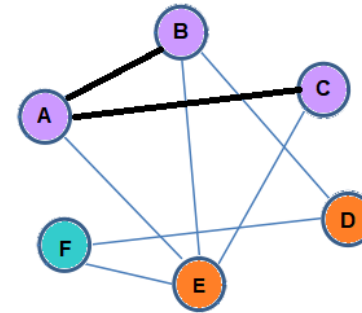
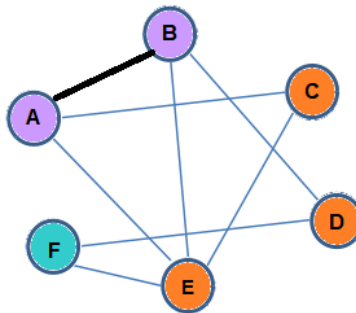
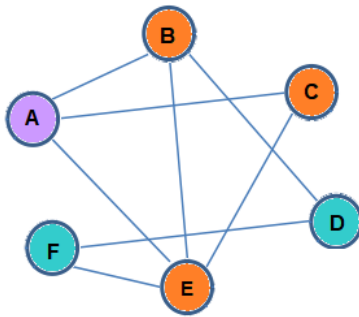
- có thể bị bỏ quên khi duyệt, nên trong giải thuật cần cân nhắc để đưa thêm các nút này vào

- Giải thuật duyệt :
 - Bước 1: Thăm nút v_s .
 - Bước 2: Tìm tập V_{AS} là các nút kề của v_s mà chưa được thăm.
 - Bước 3: Thăm lần lượt các nút trong V_{AS} .
 - Bước 4: Lặp lại giải thuật cho từng nút trong V_{AS} .
- Sử dụng CTDL trung gian để lưu các nút kề sẽ lần lượt được thăm: hàng đợi Q (các nút được lưu trước sẽ được thăm trước).

```
void BreathFirstSearch(G,  $v_s$ ) {  
    insert( $v_s$ , Q);  
    while (!isEmpty(Q)) {  
        u = remove(Q);  
        visit(u);  
        for each  $u_a$ : (adjacent to u) && (unvisited) && ( $u_a \notin Q$ )  
            // normally sorted by key value  
            insert( $u_a$ , Q);  
    }  
}
```

Duyệt theo chiều rộng

- Độ phức tạp của thủ tục: phụ thuộc vào thao tác tìm kiếm nút kề cận u_a
- Có thể đánh dấu 3 trạng thái cho các nút
 - Nút chưa được thăm và chưa nằm trong hàng đợi
 - Nút chưa được thăm nhưng đang nằm trong hàng đợi
 - Nút đã được thăm



4. Một số bài toán về đồ thị

4.1. Các bài toán tìm đường đi ngắn nhất

- Đối với đồ thị có trọng số, có một lớp các bài toán tìm đường đi ngắn nhất:
 - Tìm đường đi ngắn nhất giữa hai đỉnh
 - Tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại
 - Tìm đường đi ngắn nhất giữa mọi cặp đỉnh
- Ứng dụng:

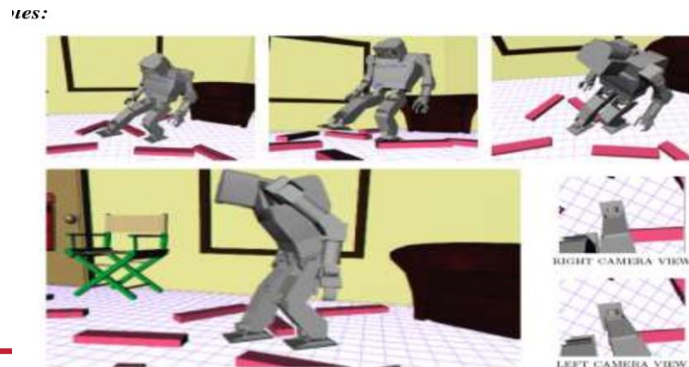
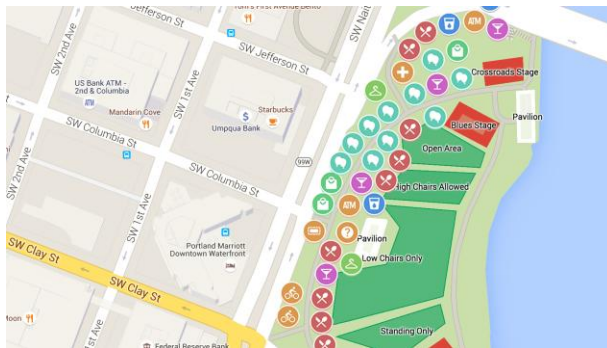
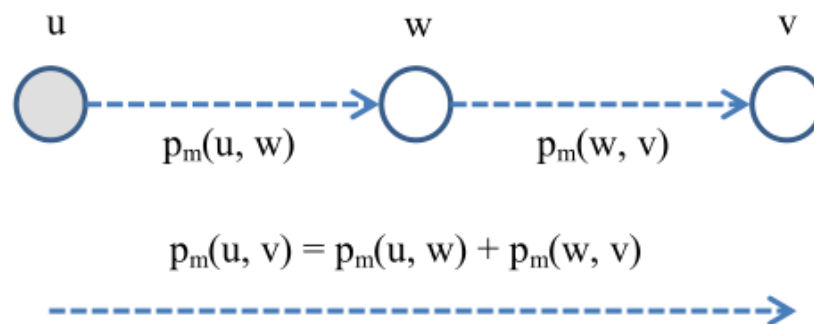


Fig. 1: Path Finding Examples

4.1. Các bài toán tìm đường đi ngắn nhất

- Tính chất (tính chất bộ phận):
 - Cho đồ thị $G(V, E)$, và hai đỉnh $u, v \in V$.
 - Gọi $p_m(u, v)$ là đường đi ngắn nhất từ u đến v .
 - Khi đó, $\forall$ đỉnh w nằm trên $p_m(u, v)$; $p(u, w)$ và $p(w, v)$ đều là các đường đi ngắn nhất.



- Chiến lược tìm đường đi ngắn nhất giữa hai đỉnh u (đỉnh nguồn) và v (đỉnh đích):
 - nếu chỉ có một đường đi giữa u và v thì đó chính là đường đi phải tìm
 - nếu không cần kiểm tra và so sánh tất cả đường đi giữa u và v
 - phương pháp duyệt theo chiều sâu bắt đầu từ nút u sẽ được sử dụng để tìm kiếm và so sánh các đường đi này

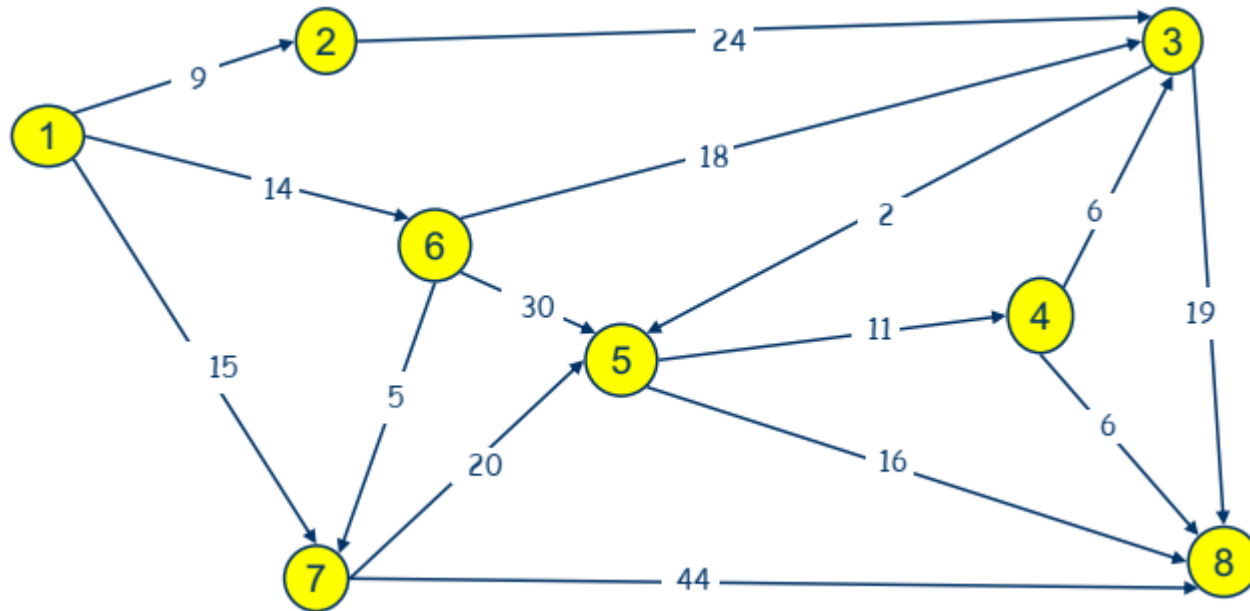
Giải thuật Dijkstra

- Tìm đường đi ngắn nhất từ một đỉnh (nút nguồn) đến các đỉnh còn lại (nút đích) của một đồ thị
- Áp dụng trên đồ thị có trọng số dương và liên thông
- Ý tưởng giải thuật:
 - Đầu vào: Cho đồ thị $G(V, E)$ với $V = v_0, v_1, \dots, v_{M-1}$ và $E = e_0, e_1, \dots, e_{N-1}$. Chọn một đỉnh bất kỳ làm nguồn, ta giả sử là đỉnh $s = v_0$.
 - Đầu ra: tập các đường đi ngắn nhất từ đỉnh nguồn s đến các đỉnh còn lại
- Gọi F là tập các nút mà đường đi ngắn nhất đã tìm được. Ban đầu ta có $F = \{s\}$.
- Nhiệm vụ chính của giải thuật là bổ sung dần các nút thích hợp vào F cho đến khi $F = V$
- Nút thích hợp ở mỗi bước là nút $v_m \in V \setminus F$:
 - v_m là nút kề của F (tồn tại ít nhất một nút $v_k \in F$ sao cho v_m là nút kề của v_k)
 - Độ dài đường đi ngắn nhất từ nút nguồn s đến nút v_m : $dist(v_m)$ có giá trị nhỏ nhất trong số các nút thuộc tập $V \setminus F$

- Triển khai:
- Với mỗi đỉnh v sẽ duy trì các thông số sau
 - $dist[v]$: Khoảng cách ngắn nhất biết được tại thời điểm hiện tại từ đỉnh nguồn s tới đỉnh v
 - $prev[v]$: Đỉnh trước của đỉnh v trên đường đi từ đỉnh nguồn s tới v
 - Duy trì một tập F chứa các đỉnh, cụm này lúc đầu chứa đỉnh xuất phát s .
 - Dần dần thêm các đỉnh vào trong cụm
 - Tại mỗi bước của giải thuật
 - Xác định đỉnh u chưa ở trong F có giá trị $dist[u]$ nhỏ nhất
 - Đưa u vào trong F
 - Cập nhật lại giá trị $dist$ của các đỉnh lân cận của u

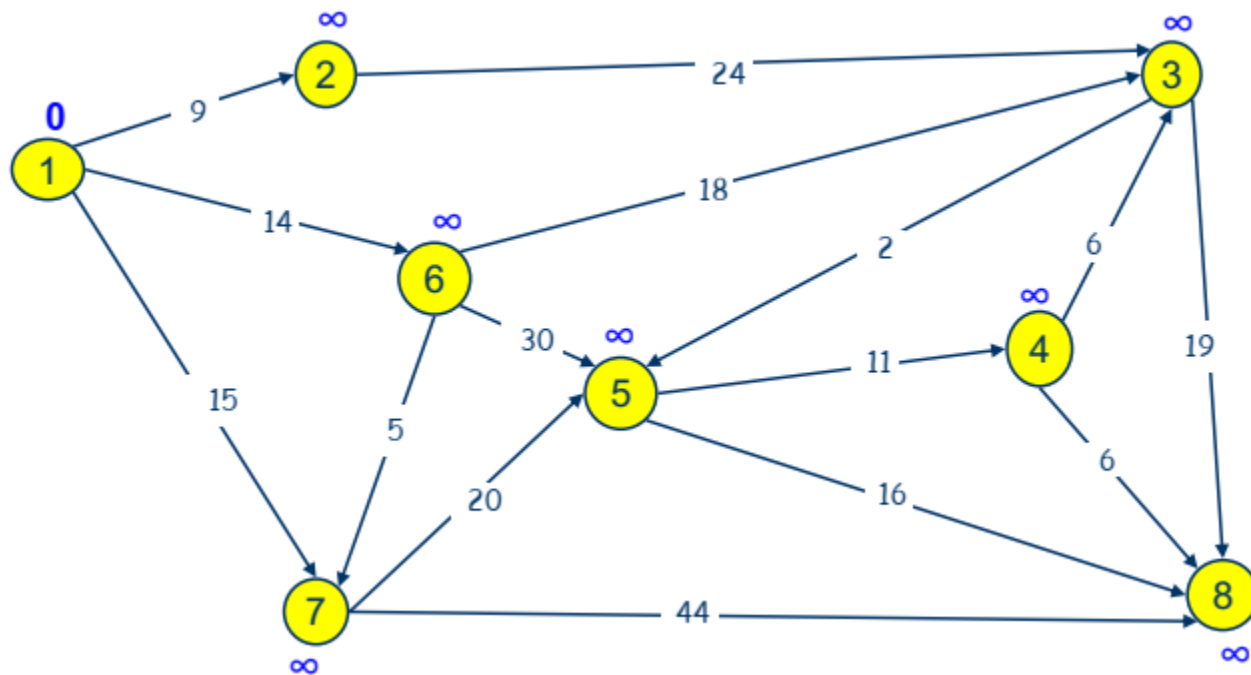
Ví dụ minh họa

- Cho đồ thị gồm các đỉnh, cạnh và các trọng số như hình vẽ
- Tìm đường đi ngắn nhất từ đỉnh số 1 tới các đỉnh còn lại



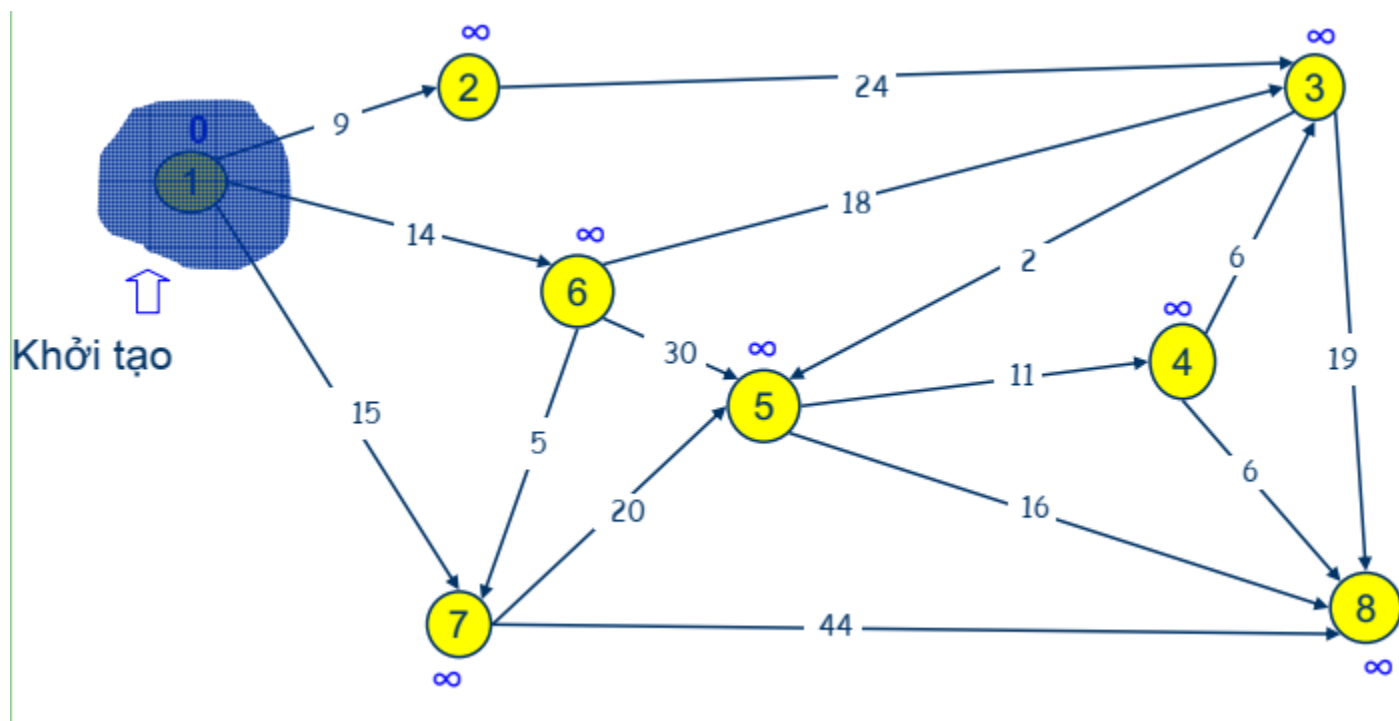
Ví dụ minh họa

- Khởi tạo giá trị *dist* cho tất cả các đỉnh
 - ∞ : giá trị chưa xác định



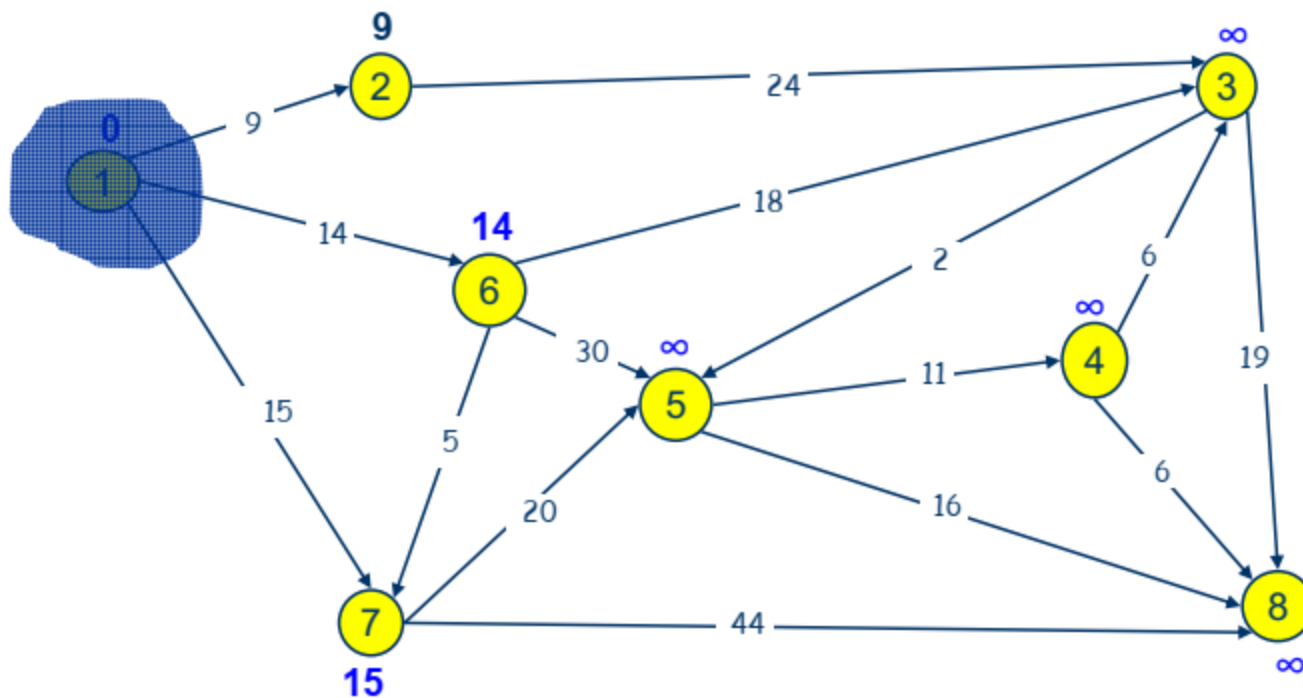
Ví dụ minh họa

- Khởi tạo tập F



Ví dụ minh họa

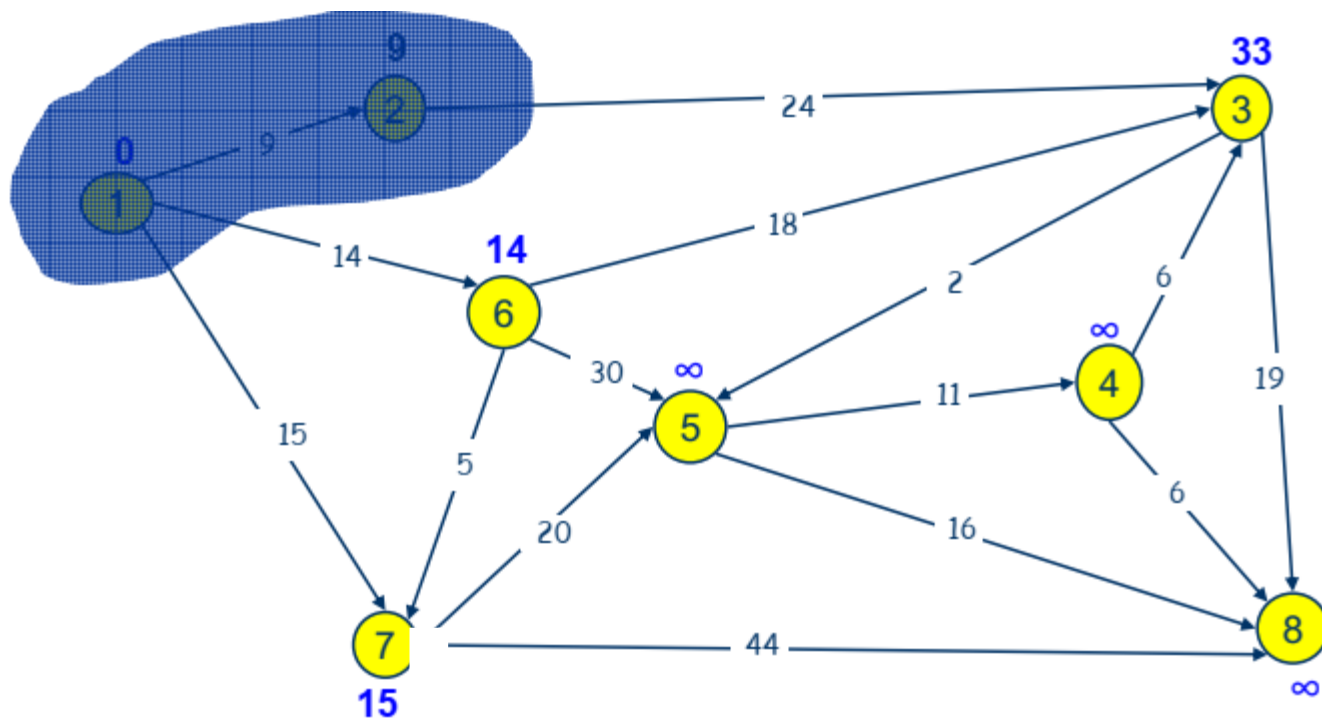
- Cập nhật giá trị dist cho các đỉnh kề của F:
 - $\text{dist}[2]=9$, $\text{dist}[6]=14$, $\text{dist}[7]=15$



Ví dụ minh họa

- Mở rộng tập F

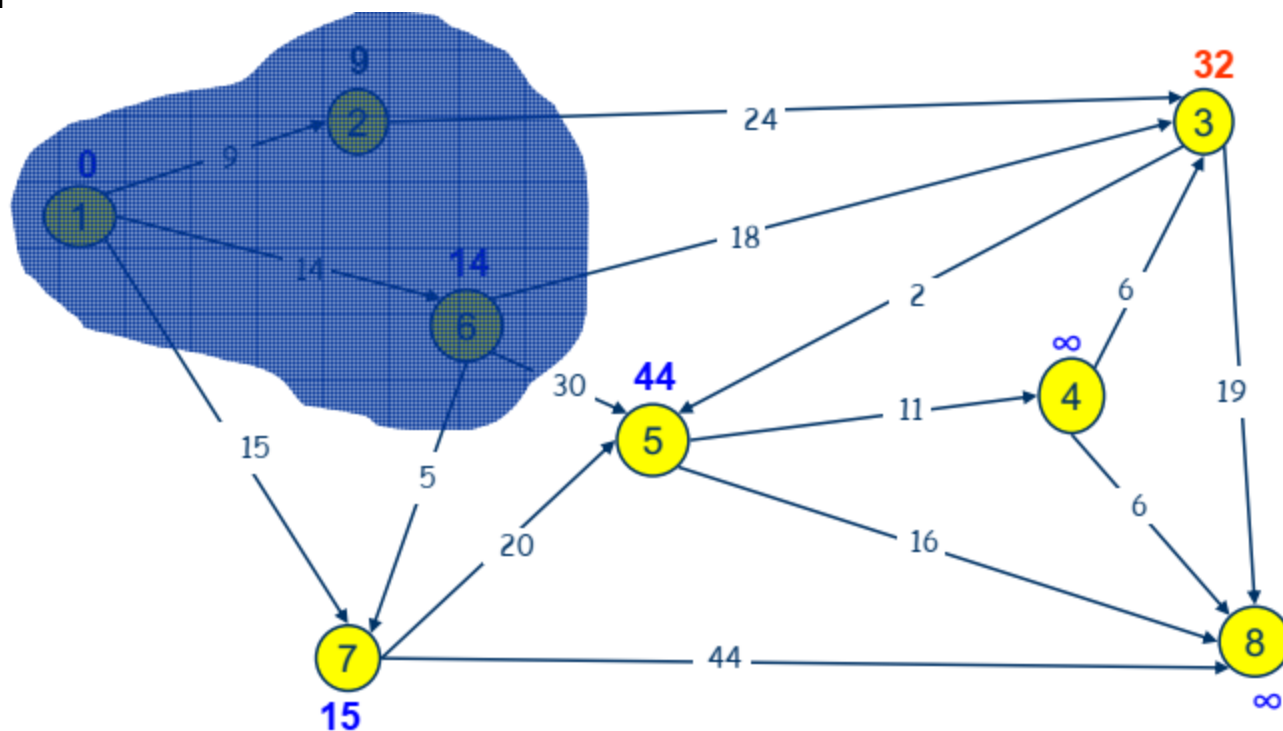
- chọn đường đi ngắn nhất $\text{dist} = 9$: từ đỉnh 1 đến đỉnh 2
- Thêm đỉnh 2 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 2: $\text{dist}[3]=33$



Ví dụ minh họa

- Mở rộng tập F

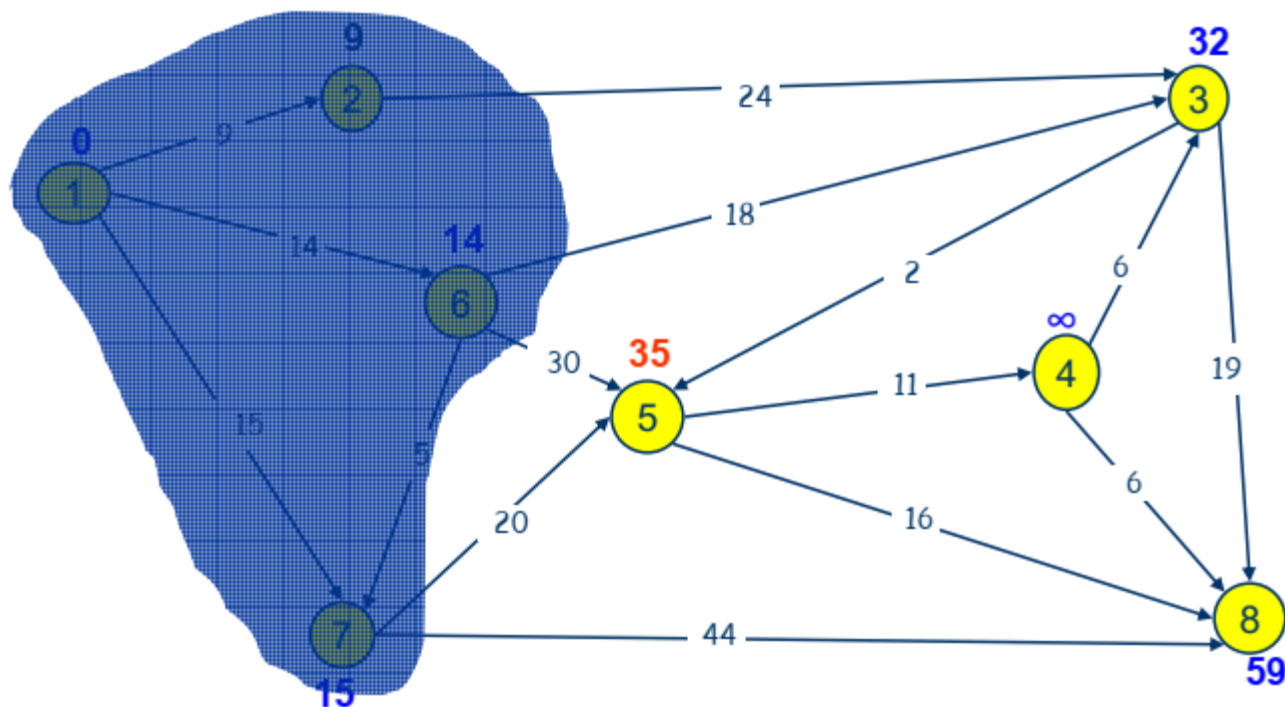
- chọn đường đi ngắn nhất $\text{dist} = 14$: từ đỉnh 1 đến đỉnh 6
- Thêm đỉnh 6 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 6: $\text{dist}[3]=33 \rightarrow 32$, $\text{dist}[7]=15$, $\text{dist}[5]=44$



Ví dụ minh họa

- Mở rộng tập F

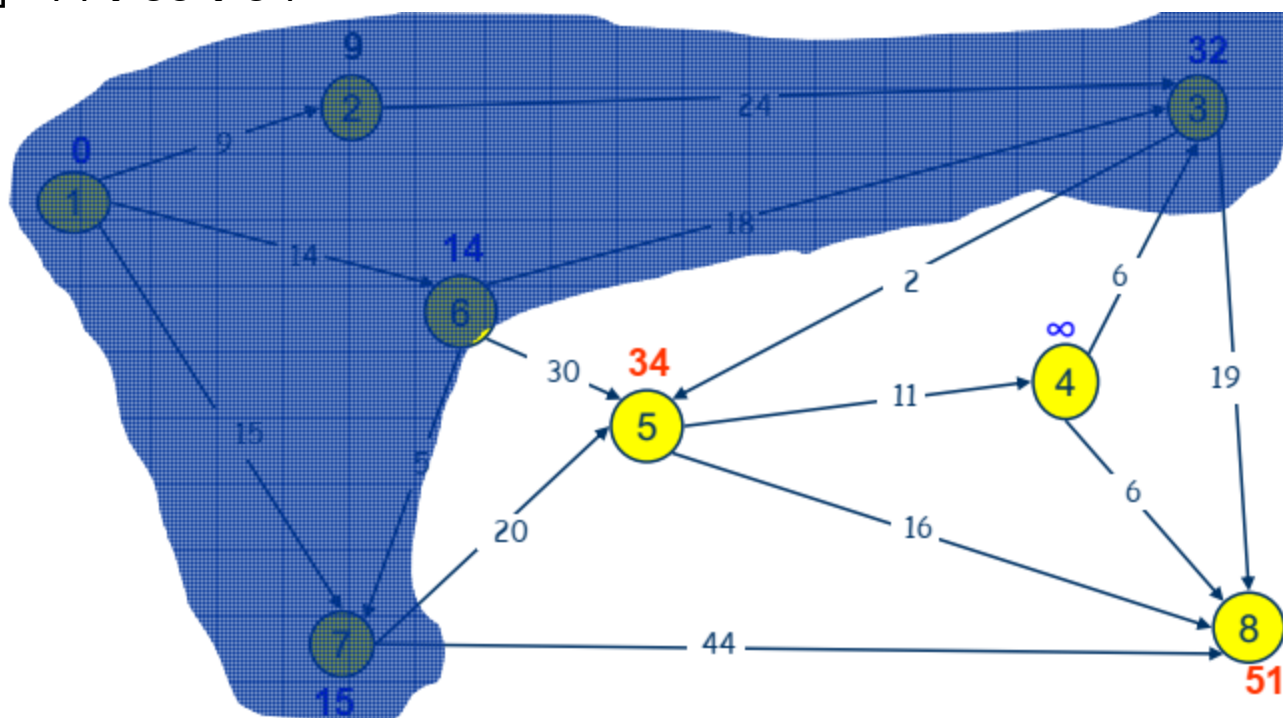
- chọn đường đi ngắn nhất $\text{dist} = 15$: từ đỉnh 1 đến đỉnh 7
- Thêm đỉnh 7 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 7: $\text{dist}[8]=59$, $\text{dist}[5]=44 \rightarrow 35$



Ví dụ minh họa

- Mở rộng tập F

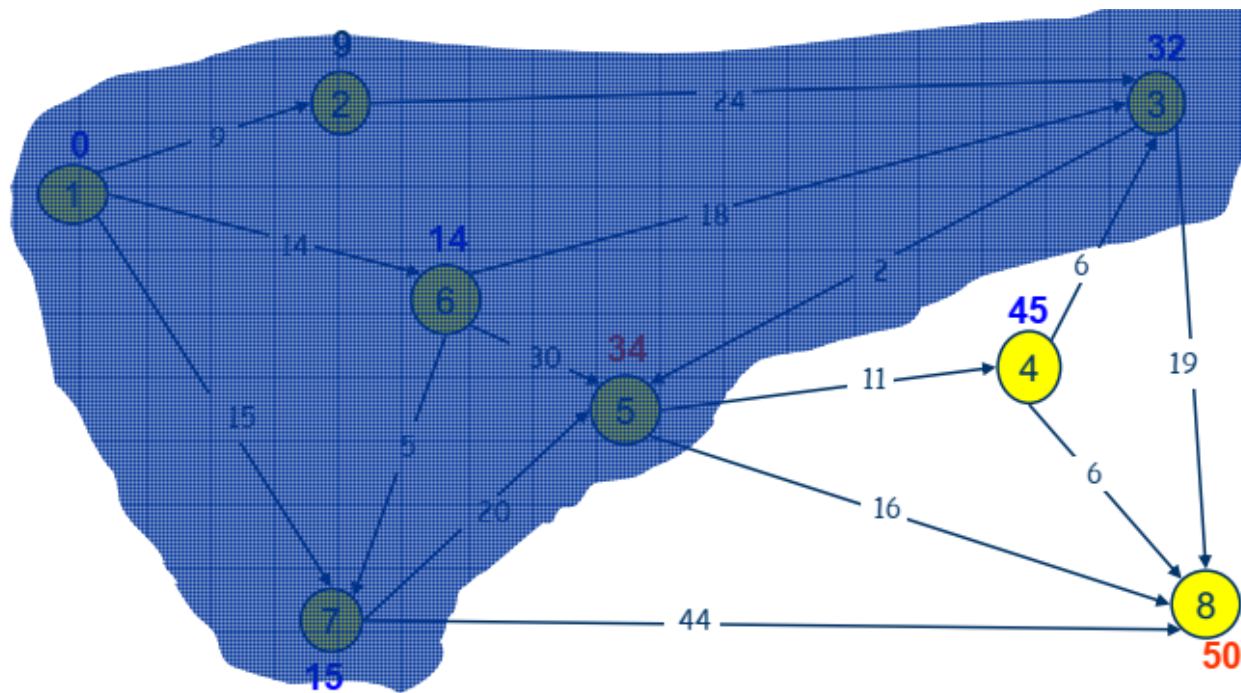
- chọn đường đi ngắn nhất $\text{dist} = 32$: từ đỉnh 1 đến đỉnh 3, đi qua đỉnh 6
- Thêm đỉnh 3 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 3: $\text{dist}[8] = 59 \rightarrow 51$, $\text{dist}[5] = 44 \rightarrow 35 \rightarrow 34$



Ví dụ minh họa

- Mở rộng tập F

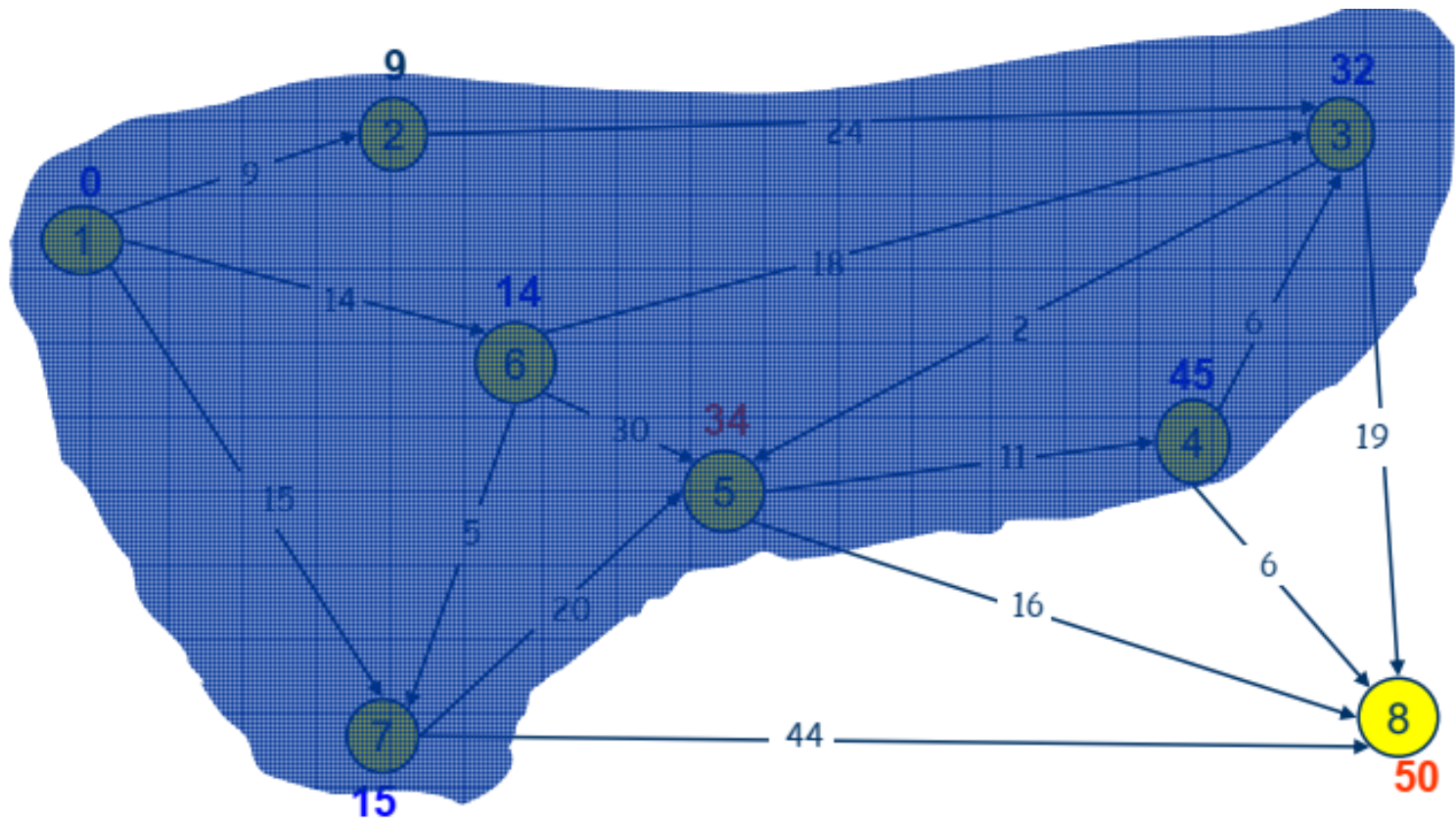
- chọn đường đi ngắn nhất $\text{dist} = 34$: từ đỉnh 1 đến đỉnh 5, đi qua đỉnh 6 và 3
- Thêm đỉnh 5 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 5: $\text{dist}[8] = 59 \rightarrow 51 \rightarrow 50$, $\text{dist}[4] = 45$



Ví dụ minh họa

- Mở rộng tập F

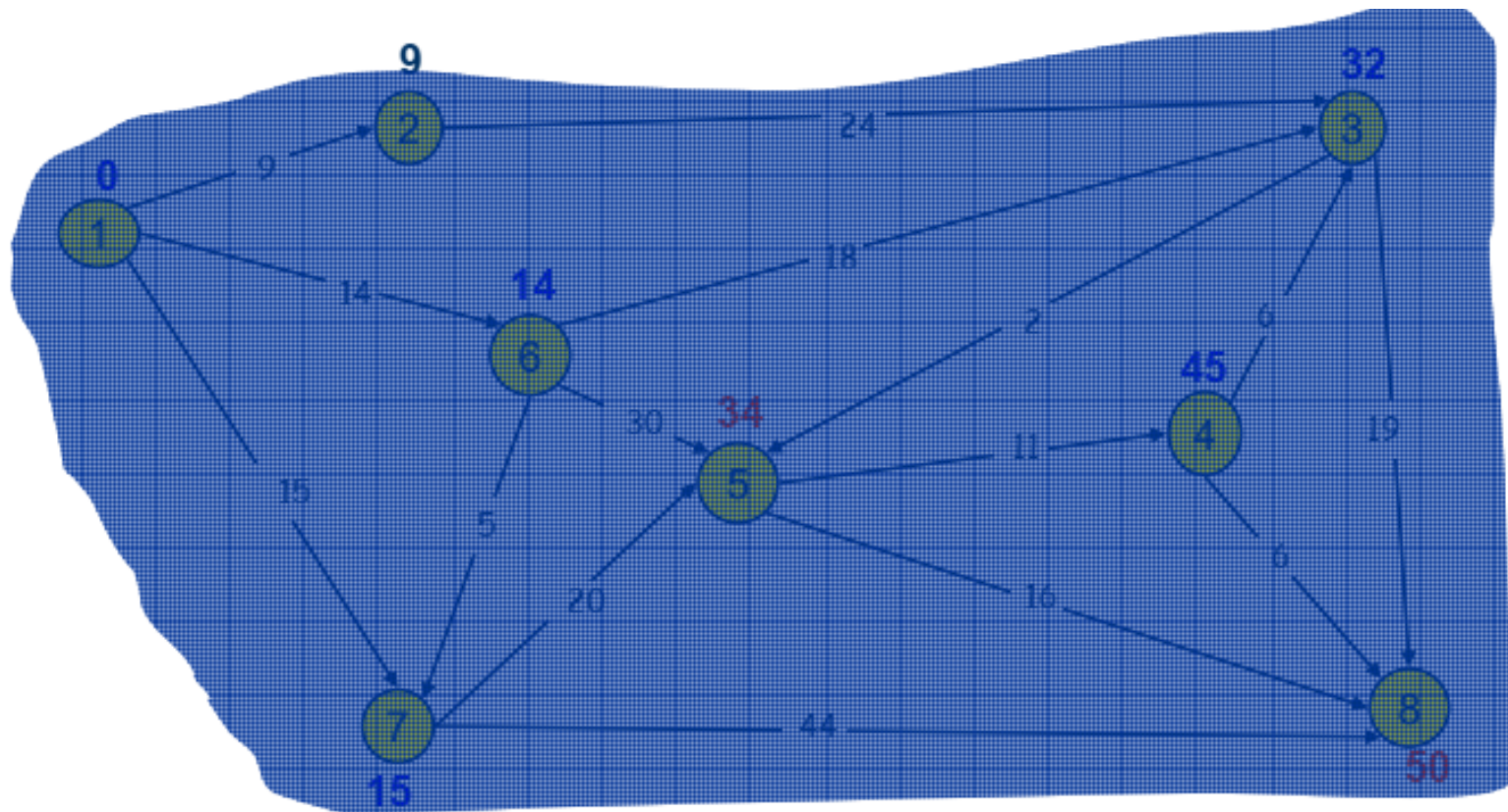
- chọn đường đi ngắn nhất $\text{dist} = 45$: từ đỉnh 1 đến đỉnh 4, đi qua đỉnh 6 – 3 – 5
- Thêm đỉnh 4 vào F
- Cập nhật giá trị dist cho các đỉnh kề của đỉnh 4: $\text{dist}[8] = 59 \rightarrow 51 \rightarrow 50 \rightarrow 50$



Ví dụ minh họa

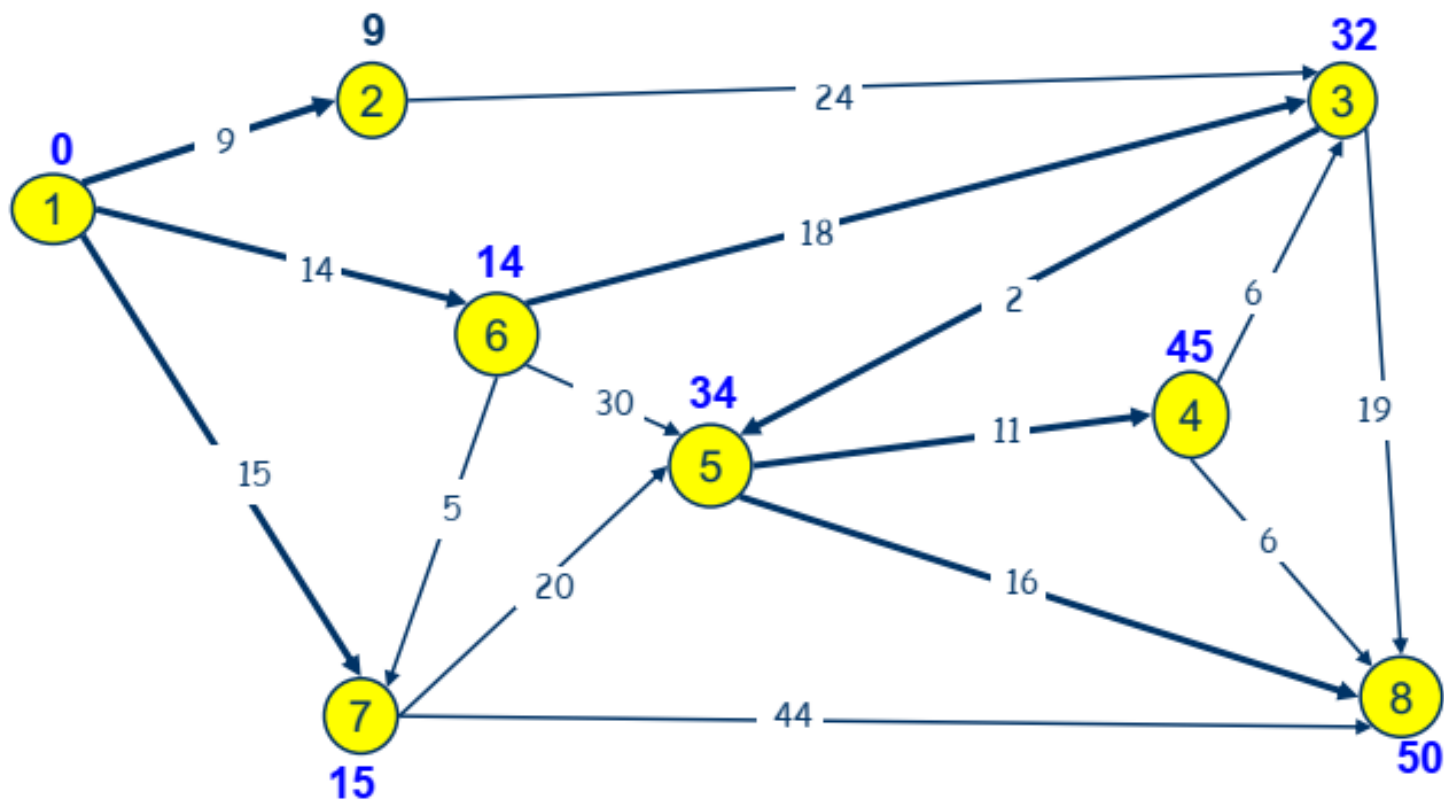
- Mở rộng tập F

- chọn đường đi ngắn nhất $\text{dist} = 50$: từ đỉnh 1 đến đỉnh 8, đi qua đỉnh 6 – 3 – 5
- Thêm đỉnh 8 vào F. $F=V \rightarrow$ Dừng



Ví dụ minh họa

- Tập hợp đường đi ngắn nhất từ đỉnh 1 tới các đỉnh còn lại.



- Cài đặt

```
F = {v0}; dist[0] = 0;
for (i=1; i<N;i++) {
    if (v0, vi) ∈ E {
        dist[i] = |v0,vi| ; prev[i]=v0;
    } else dist[i] = ∞;
}
for (i=1; i<N; i++){
    Tìm vm ∈ V\F: dist[m]= MIN(dist[k] ∀ vk ∈ V\F) ;
    F = F ∪ vm;
    for each vk ∈ V\F
        if ((vm, vk) ∈ E) && (dist[k]>dist[m]+|vm,vk|) {
            dist[k] = dist[m] + |vm, vk| ;
            prev[k] = vm ;
        }
}
```

$|v0, vk|$: độ dài cạnh $(v0, vk)$ (trọng số của cạnh)

Giải thuật Bellman-Ford

- Tìm “các” đường đi ngắn nhất từ một nút nguồn đến các nút còn lại.
- Có độ phức tạp lớn hơn giải thuật Dijkstra, nhưng bù lại, có thể áp dụng cho cả hai loại đồ thị có trọng số âm và trọng số dương.
- Tự tìm hiểu

4.2. Bài toán tìm chu trình của đồ thị

- Chu trình: đường đi khép kín (tức là đỉnh đầu và đỉnh cuối trùng nhau) và không có cạnh nào xuất hiện quá một lần
- Một số bài toán con:
 - a. Kiểm tra sự tồn tại của chu trình trong đồ thị
 - b. Kiểm tra sự tồn tại của chu trình xuất phát từ một đỉnh/cạnh cho trước.
 - c. Tìm tất cả các chu trình trong đồ thị
- Kiểm tra chu trình
 - Áp dụng chiến lược duyệt đồ thị theo chiều sâu

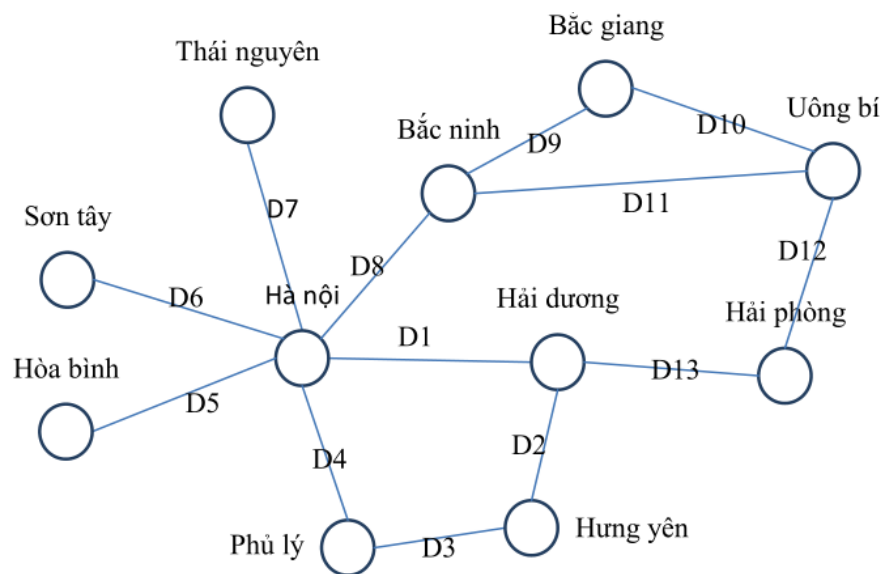
4.2. Bài toán tìm chu trình của đồ thị

- Kiểm tra sự tồn tại của chu trình xuất phát từ một đỉnh cho trước
 - Cho đồ thị $G(V, E)$ với $V = v_0, v_1, \dots, v_{M-1}$, $E = e_0, e_1, \dots, e_{N-1}$, và đỉnh ban đầu $s \in V$
 - Kiểm tra xem có tồn tại một chu trình mà chứa s hay không ?
- Ý tưởng giải thuật:
 - Duyệt theo chiều sâu, bắt đầu từ s
 - Giả sử quá trình kiểm tra đang đến nút c . Gọi tập các đỉnh hiện tại kề với c là A_c .
 - Nếu A_c rỗng $\rightarrow$ không có chu trình;
 - Nếu tồn tại $v_i \in A_c : v_i = s \rightarrow$ có tồn tại chu trình;
 - Nếu $A_c \neq \emptyset$ và $\forall v_i \in A_c$ đều có $v_i \neq s \rightarrow$ Lặp lại giải thuật cho từng $v_i \in A_c$.

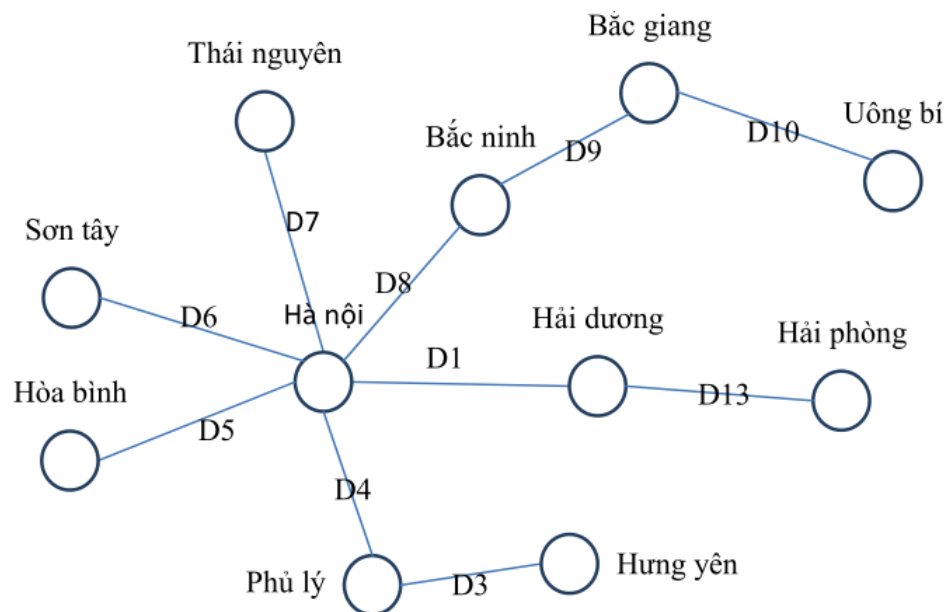
4.3. Bài toán tìm cây khung có giá cực tiểu

• Cây khung (spanning tree)

- Cây khung của một đồ thị là phần có cấu trúc cây (không xuất hiện chu trình) mà có số cạnh lớn nhất của đồ thị đó.
- Với một cây khung, nếu ta thêm vào bất kỳ một cạnh nào khác của đồ thị thì nó không còn là cấu trúc cây nữa (bắt đầu xuất hiện chu trình)



Cây khung từ đồ thị



4.3. Bài toán tìm cây khung có giá cực tiểu

- Cây khung của một đồ thị $G(V, E)_{(M \text{ nút}, N \text{ cạnh})}$ là một đồ thị con liên thông T
 - Bao gồm M nút và không có chu trình (tức là nó sẽ có $M-1$ cạnh).
- Khi $N \geq M-1$ thì G có thể có một hoặc nhiều cây khung.
- Bài toán tìm cây khung có giá cực tiểu (cây khung cực tiểu)
 - Áp dụng cho đồ thị G là đồ thị liên thông, vô hướng và có trọng số (G sẽ có ít nhất một cây khung).
 - Tìm cây khung T_m có tổng giá trị các trọng số trên các cạnh là nhỏ nhất trong số tất cả các cây khung của G .
- $\Leftrightarrow$ Bài toán cần chọn ra $M-1$ giá trị (ứng với trọng số của $M-1$ cạnh) từ N giá trị sao cho tổng của $M-1$ giá trị này là nhỏ nhất và không tạo ra chu trình
 - Giải thuật Kruskal
- $\Leftrightarrow$ Bài toán loại bỏ đúng $N-M+1$ cạnh, sao cho tổng các cạnh bị loại bỏ có giá trị lớn nhất và các cạnh còn lại không tạo ra chu trình.
 - Giải thuật Dijkstra

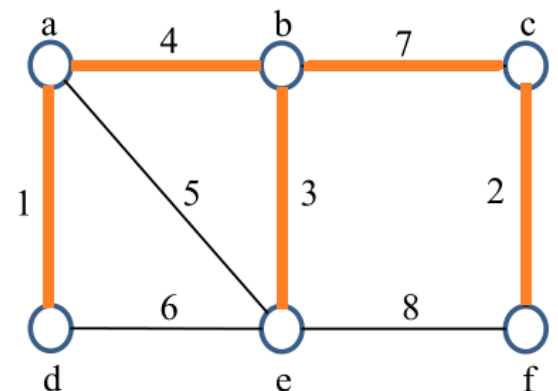
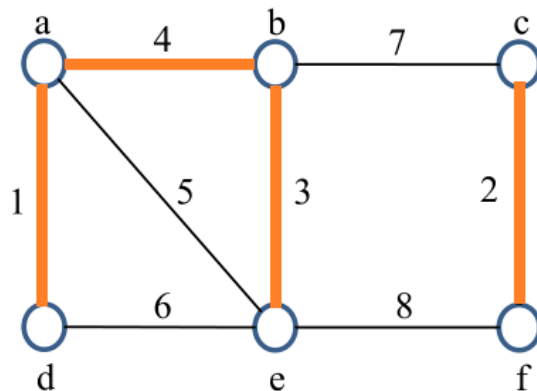
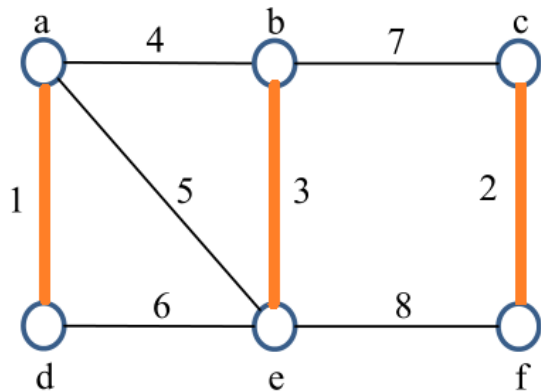
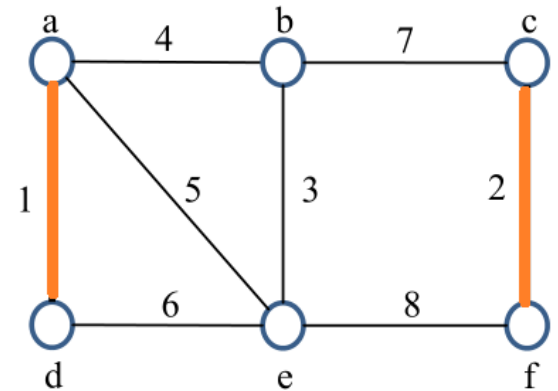
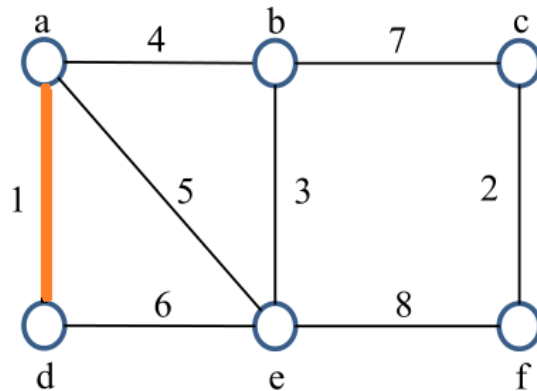
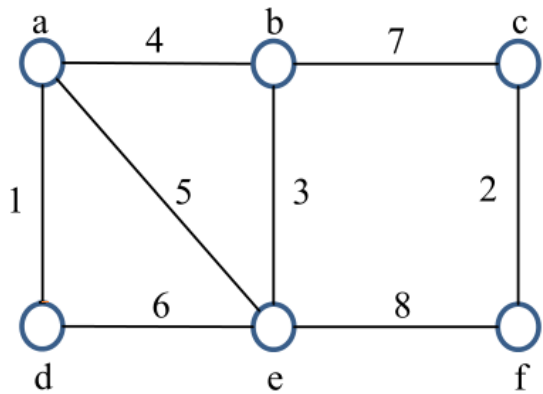
Giải thuật Kruskal

- Xuất phát từ cây khung rỗng (chỉ có các đỉnh mà chưa có cạnh) bổ sung dần các cạnh có giá trị nhỏ nhất có thể vào (không tạo thành chu trình), cho đến khi tạo thành cây khung đầy đủ (đúng $M-1$ cạnh).
- Các cạnh của đồ thị sẽ được sắp xếp theo giá trị trọng số theo thứ tự tăng dần
- Quá trình chọn các cạnh sẽ lần lượt từ nhỏ đến lớn
- Cạnh được chọn thỏa mãn điều kiện không tạo ra chu trình với các cạnh đã được chọn trước đó

```
T =  $\emptyset$ ;  
S = tập các cạnh được sắp xếp của E;  
    //giả sử: S = (s0, s1, ..., sN-1)  
i = 0;  
while ((i < N) và (|T| < M-1)){  
    Nếu si không tạo ra chu trình thì: T = T U si;  
    i++;  
}
```

Giải thuật Kruskal

- Minh họa

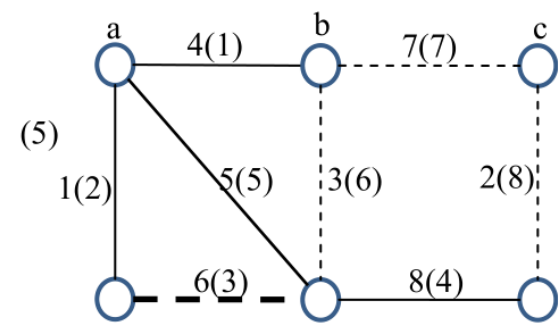
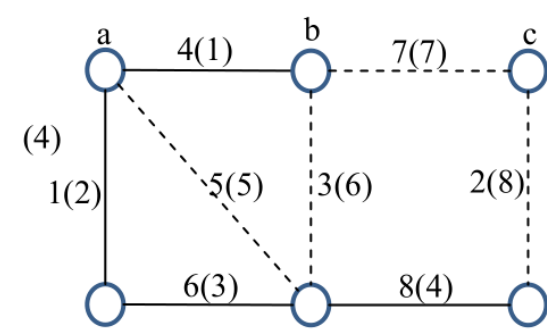
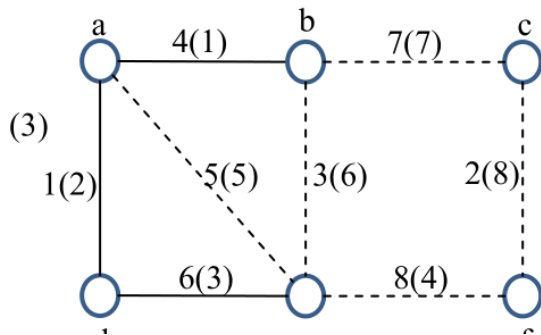
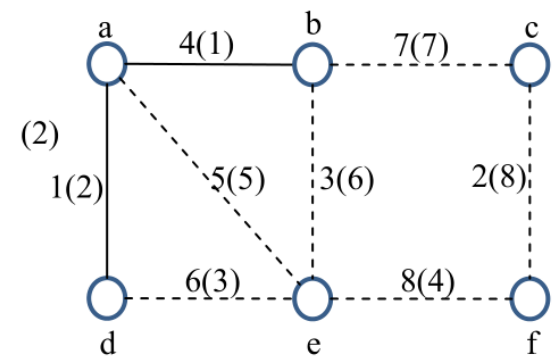
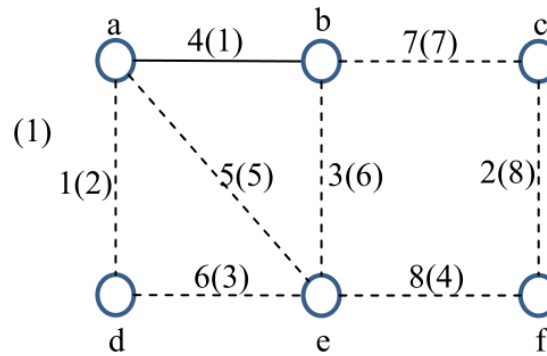
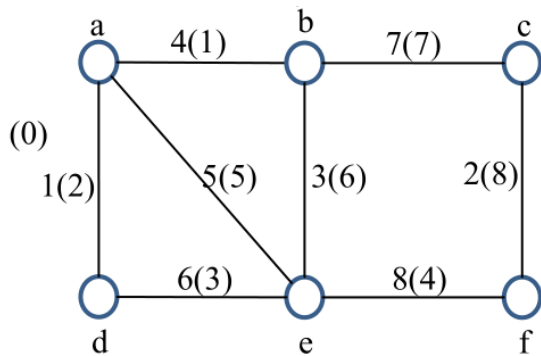


- Bắt đầu từ đồ thị ban đầu, tiến hành loại bỏ dần các cạnh có giá trị lớn nhất có thể (những cạnh mà gây ra chu trình), cho đến khi còn lại cây khung thì đó sẽ là cây khung cực tiểu.
 - các cạnh của đồ thị cần được sắp xếp, giải thuật sắp xếp tốt nhất có độ phức tạp $N \cdot \log(N)$
- Cải tiến để tránh thao tác sắp xếp
 - Xuất phát từ cây khung rỗng ban đầu, tiến hành bổ sung lần lượt từng cạnh của đồ thị theo thứ tự bất kì (không cần sắp xếp), cho đến khi đủ thành cây khung.
 - Mỗi lần bổ sung một cạnh, kiểm tra xem có tạo thành chu trình không.
 - Nếu chu trình được tạo thành, tìm cạnh có trọng số lớn nhất trong chu trình và loại bỏ nó ra.

```
T = ∅;  
for (i = 0; i < N; i++) {  
    T = T ∪ ei;  
    if (ei tạo ra chu trình ci) {  
        Tìm em ∈ ci : em là cạnh max trong ci;  
        Loại bỏ em;  
    }  
}
```

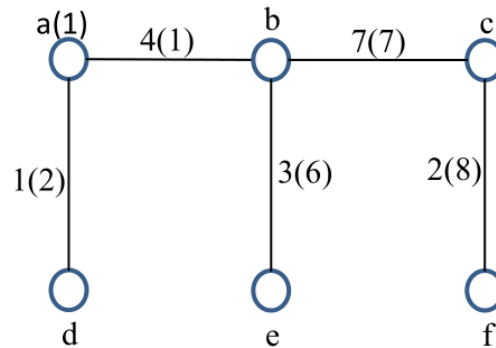
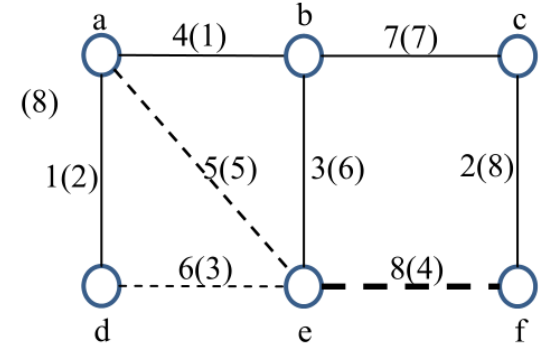
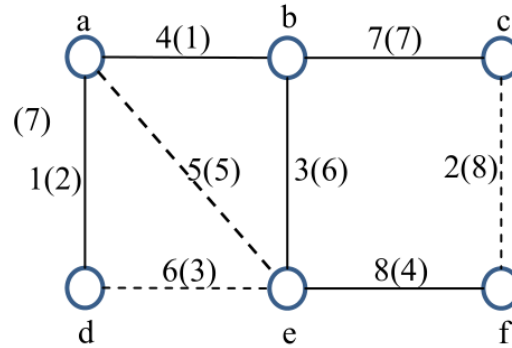
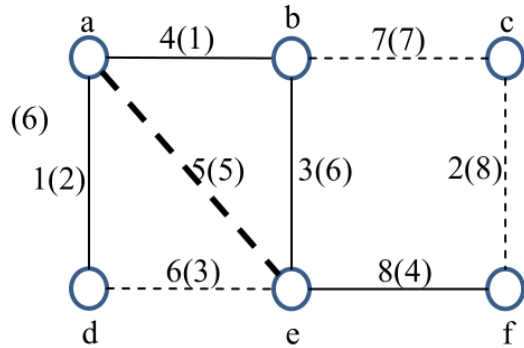
Giải thuật Dijkstra

- Minh họa



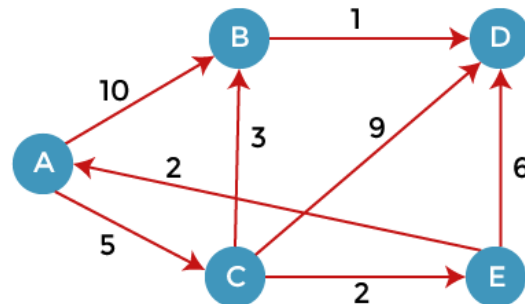
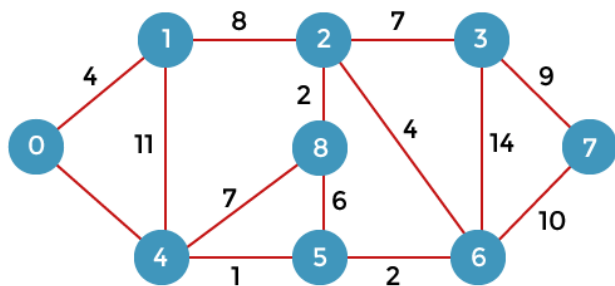
Giải thuật Dijkstra

- Minh họa

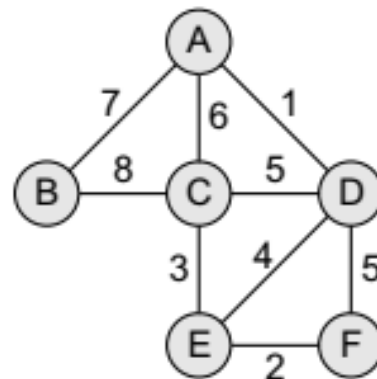
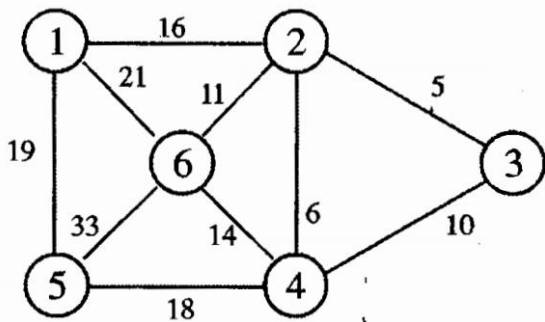


Bài tập

- Sử dụng giải thuật Dijkstra tìm đường đi ngắn nhất từ đỉnh 1 đến các đỉnh còn lại trong đồ thị



- Tìm cây khung cực tiểu của đồ thị



4.2. Bài toán tìm chu trình của đồ thị

- Giải thuật

```
bool CycleDetection(G, s, c){  
    Ac = Tập đỉnh kề của c;  
    if (Ac là rỗng)  
        return false;  
    else  
        if ( $\exists v_i \in Ac \ \&\& \ v_i == s$ ) return true;  
        if ( $\exists v_i \in Ac: \text{CycDetection}(G, s, v_i) == \text{true}$ )  
            return true;  
        else  
            return false;  
}
```